

Granite 4.0-H 350M: Nano Model with Excellent Tool Calling: Run On-Device



Fahd Mirza
48.5K subscribers

Join

Subscribe

42



Share

Ask

Download



855 views Oct 28, 2025 #granitenano #granite4

This video tests MiniMax M2, which redefines efficiency for agents.

This video is sponsored by <https://gradient.network>, Parallax, a distributed model serving framework that lets you build your own AI cluster anywhere.

Hugging Face Search models, datasets, Models Datasets Spaces Docs Pricing Log In Sign Up

ibm-granite's Collections

Granite 4.0 Nano Language Models	Granite 4.0 Nano Language Mo... updated about 5 hours ...	▲ Upvote 33
Granite 4.0 Language Models	ibm-granite/granite-4.0-h-1b Text Generation · 1B · Updated 5 days ago · 36 · 31	29
Granite Docling		Share collection
Granite 3.3 Language Models	ibm-granite/granite-4.0-h-1b-base Text Generation · 1B · Updated 5 days ago · 33 · 20	View history
Granite Speech		Collection guide
Granite 3.2 Language Models	ibm-granite/granite-4.0-h-350m Text Generation · 0.3B · Updated 5 days ago · 802 · 24	Browse collections
Granite Vision Models		
Granite 3.1 Language	ibm-granite/granite-4.0-h-350m-base Text Generation · 0.3B · Updated 5 days ago · 20 · 16	

3.3 in 8B Thinking Granite IBM Install & Test 17:21

Granite 3.3 8B Instruct: IBM's Thinking Model - Install and Test Locally
Fahd Mirza · 1.2K views · 6 months ago
This video locally installs Granite-3.3-8B-Instruct which is fine-tuned for improved reasoning and instruction-following capabilities. 🍌 Get 50% Discount on any A6000 or A5000 GPU rental...

GRANITE MICRO IBM 10:02

Granite 4 Micro: Hyper-Efficient, High-Performance Hybrid AI for Enterprise
Fahd Mirza · 3.2K views · 3 weeks ago
This video locally installs and tests Granite-4.0-Micro is a 3B parameter long-context instruct model. 🍌 Access ChatGPT 5, Perplexity, Claude and 500 more LLMs & tools with a single subscription...

← → ↻ <https://huggingface.co/ibm-granite/granite-4.0-h-350m> 150%

Hugging Face 🔍 Search models, datasets, [Models](#) [Datasets](#) [Spaces](#) [Docs](#) [Pricing](#) [Log In](#) [Sign Up](#)

[ibm-granite/granite-4.0-h-350m](#) 📄 like 24 Follow [IBM Granite](#) 3.25k

🔖 Text Generation 🧠 Transformers ⚙️ Safetensors granitemoe hybrid language granite-4.0 conversational 📄 arxiv:0000.00000

📄 License: apache-2.0

📄 Model card 📄 Files 📄 xet 🧑 Community

⋮ ⚙️ Train 🚀 Deploy 🖨️ Use this model

Granite-4.0-H-350M

Model Summary: Granite-4.0-H-350M is a lightweight instruct model finetuned from *Granite-4.0-H-350M-Base* using a combination of open source instruction datasets with permissive license and internally collected synthetic datasets. This model is developed using a diverse set of techniques including supervised finetuning, reinforcement learning, and model merging.

- **Developers:** Granite Team, IBM

Downloads last month
802

⚙️ **Safetensors** ⓘ

Model size 0.3B params Tensor type BF16

🗨️ Chat template ➔ Files info

⚡ **Inference Providers** NEW

🗨️ Text Generation

This model isn't deployed

```
ubuntu@0054-dsm2-sma6-prmx70017: ~  
ubuntu@0054-dsm2-sma6-prmx70017:~$ echo "CPU: $(lscpu | grep 'Model name' | cut -d: -f2 | xargs)" && free  
-h | grep '^Mem:' | awk '{print "RAM: "$2}'  
CPU: Intel(R) Xeon(R) CPU E5-2683 v4 @ 2.10GHz  
RAM: 47Gi  
ubuntu@0054-dsm2-sma6-prmx70017:~$
```

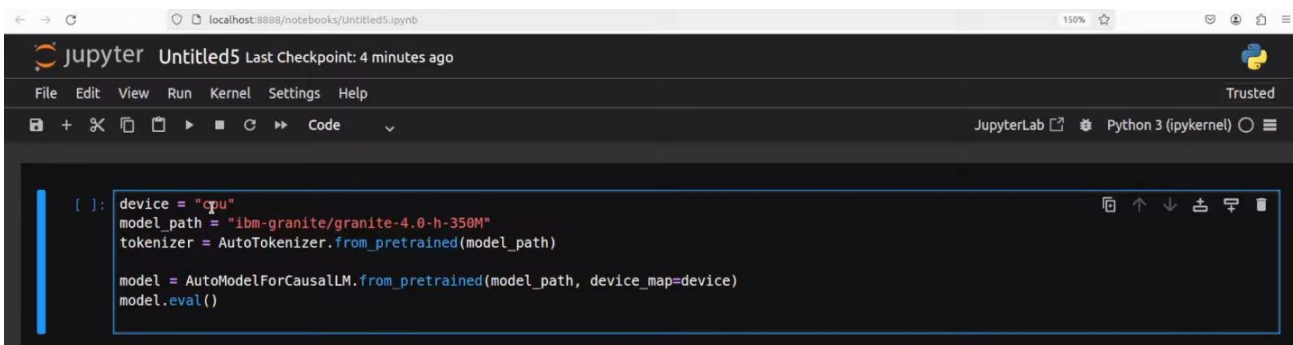
```
+-----+  
ubuntu@0054-dsm2-sma6-prmx70017:~$ conda create -n ai python=3.11 -y && conda activate ai  
Collecting package metadata (current_repodata.json): -
```

```
(ai) ubuntu@0054-dsm2-sma6-prmx70017:~$ pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121  
pip install einops timm pillow  
pip install git+https://github.com/huggingface/transformers  
Looking in indexes: https://download.pytorch.org/whl/cu121  
Collecting torch  
Using cached https://download.pytorch.org/whl/cu121/torch-2.5.1%2Bcu121-cp311-cp311-linux_x86_64.whl (780.5 MB)
```

```
validator-0.1.4 rfc3986-validator-0.1.1 rfc3987-syntax-1.1.0 rpds-py-0.28.0 send2trash-1.8.3 six-1.17.0 soupsieve-2.8 terminado-0.18.1 tinycss2-1.4.0 tornado-6.5.2 tzdata-2025.2 uri-template-1.3.0 webcolors-24.11.1 webencodings-0.5.1 websocket-client-1.9.0 widgetsnbextension-4.0.14  
(ai) ubuntu@0054-dsm2-sma6-prmx70017:~$ hf auth login  
User is already logged in.  
(ai) ubuntu@0054-dsm2-sma6-prmx70017:~$
```

```
(ai) ubuntu@0054-dsm2-sma6-prmx70017:~$ hf download ibm-granite/granite-4.0-h-350m \
--local-dir checkpoints
Fetching 12 files: 0%|
Download 'model.safetensors' to 'checkpoints/.cache/huggingface/download/xGOKKLRSLIhH692hSVvI1-gpoa8=.5e8e1b9957216467f080a186b13ef8886f23a6da273f55b8287f41de54faafc7.incomplete'
Download 'config.json' to 'checkpoints/.cache/huggingface/download/8_PA_wEVG1Va2goH2H4K00qpVvY=.35e00a1ad4b1fea2a18d40b4880291379f6c5947.incomplete'
Download 'generation_config.json' to 'checkpoints/.cache/huggingface/download/3EVKVgg0ldJcKSsGjSdoUCN1AyQ=.2eed7ca2d26ec1a753b8800e0bae20c824e8b015.incomplete'
Download 'model.sig' to 'checkpoints/.cache/huggingface/download/C_dgUVw3BLphUuJohz45xNphUes=.280843dc1aa71233eae9abe72a59dbd1e92d70fb.incomplete'
Download 'README.md' to 'checkpoints/.cache/huggingface/download/Xn7B-BWUG0ee2Y6hCZtEhtFu4BE=.36a47209eb04a75f3cdd242e352dac4145e5490e.incomplete'
Download '.gitattributes' to 'checkpoints/.cache/huggingface/download/wPaCKH-WbT7GsmxMKKRnZTV4nSM=.a6344aac8c09253b3b630fb776ae94478aa0275b.incomplete'
config.json: 1.72kB [00:00, 3.39MB/s]
Download complete. Moving file to checkpoints/config.json
generation_config.json: 100%| 147/147 [00:00<00:00, 281kB/s]
Download complete. Moving file to checkpoints/generation_config.json
Download 'merges.txt' to 'checkpoints/.cache/huggingface/download/PtHk0z_I45atnj23IIRhTExwT3w=.354558edc
bd64ca7abd407b8be3d5d09d39d781.incomplete'
README.md: 33.2kB [00:00, 40.4MB/s]
Download complete. Moving file to checkpoints/README.md
Downloading 'chat_template.jinja' to 'checkpoints/.cache/huggingface/download/yIUQ6Jg7XpBWxEkozaEkS2ih05g=.
```

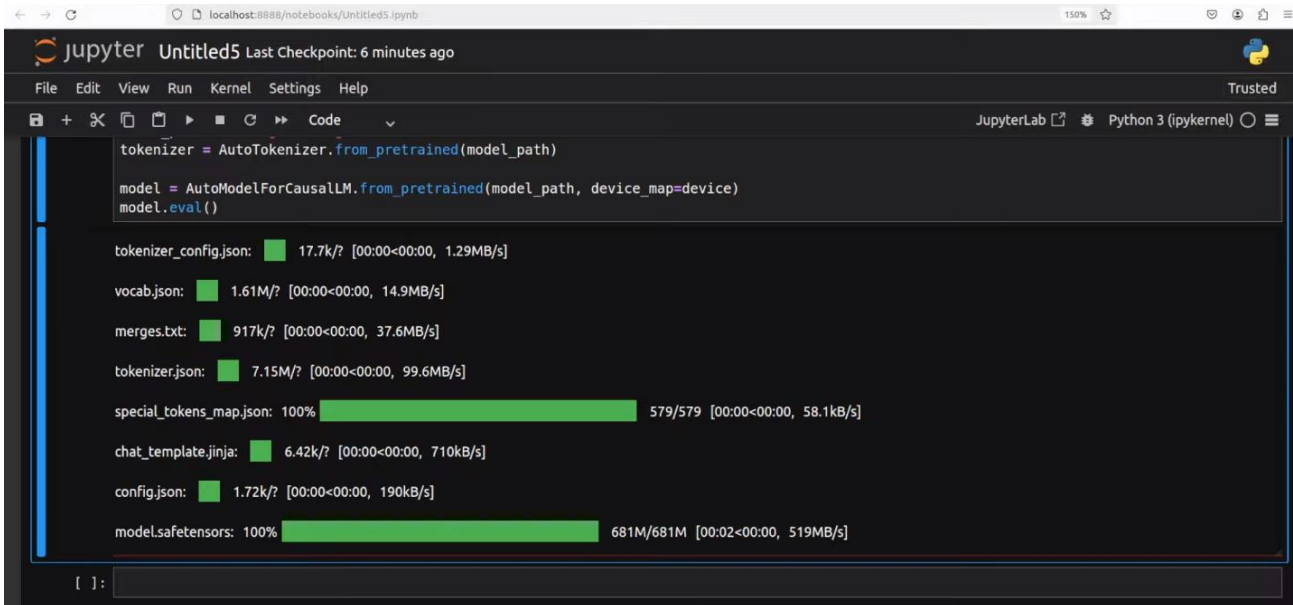
```
(ai) ubuntu@0054-dsm2-sma6-prmx70017:~$ conda install -c conda-forge --override-channels notebook -y
conda install -c conda-forge --override-channels ipywidgets -y
jupyter notebook
```



The screenshot shows the Jupyter Notebook interface with a single code cell. The code defines a device, a model path, a tokenizer, and a model, then evaluates the model.

```
[ ]: device = "cpu"
model_path = "ibm-granite/granite-4.0-h-350M"
tokenizer = AutoTokenizer.from_pretrained(model_path)

model = AutoModelForCausalLM.from_pretrained(model_path, device_map=device)
model.eval()
```



The screenshot shows the Jupyter Notebook interface with the same code cell as before, but now it displays the progress of loading the model components. The progress bars are green, indicating successful completion.

```
[ ]: tokenizer = AutoTokenizer.from_pretrained(model_path)

model = AutoModelForCausalLM.from_pretrained(model_path, device_map=device)
model.eval()

tokenizer_config.json: 17.7k/? [00:00<00:00, 1.29MB/s]
vocab.json: 1.61M/? [00:00<00:00, 14.9MB/s]
merges.txt: 917k/? [00:00<00:00, 37.6MB/s]
tokenizer.json: 7.15M/? [00:00<00:00, 99.6MB/s]
special_tokens_map.json: 100% 579/579 [00:00<00:00, 58.1kB/s]
chat_template.jinja: 6.42k/? [00:00<00:00, 710kB/s]
config.json: 1.72k/? [00:00<00:00, 190kB/s]
model.safetensors: 100% 681M/681M [00:02<00:00, 519MB/s]
```

```
tokenizer = AutoTokenizer.from_pretrained(model_path)

model = AutoModelForCausalLM.from_pretrained(model_path, device_map=device)
model.eval()

[2]: GraniteMoeHybridForCausalLM(
  (model): GraniteMoeHybridModel(
    (embed_tokens): Embedding(100352, 768, padding_idx=100256)
    (layers): ModuleList(
      (0-9): 10 x GraniteMoeHybridDecoderLayer(
        (block_sparse_moe): GraniteMoeHybridMoE(
          (activation): SiLUActivation()
          (input_linear): GraniteMoeHybridParallelExperts()
          (output_linear): GraniteMoeHybridParallelExperts()
          (router): GraniteMoeHybridTopKGating(
            (layer): Linear(in_features=768, out_features=0, bias=False)
          )
        )
        (input_layernorm): GraniteMoeHybridRMSNorm((768,), eps=1e-05)
        (post_attention_layernorm): GraniteMoeHybridRMSNorm((768,), eps=1e-05)
        (shared_mlp): GraniteMoeHybridMLP(
          (activation): SiLUActivation()
          (input_linear): Linear(in_features=768, out_features=4096, bias=False)
```

```
tokenizer = AutoTokenizer.from_pretrained(model_path)

model = AutoModelForCausalLM.from_pretrained(model_path, device_map=device)
model.eval()

(input_linear): Linear(in_features=768, out_features=4096, bias=False)
(output_linear): Linear(in_features=2048, out_features=768, bias=False)
)
(mamba): GraniteMoeHybridMambaLayer(
  (act): SiLUActivation()
  (conv1d): Conv1d(1792, 1792, kernel_size=(4,), stride=(1,), padding=(3,), groups=1792)
  (in_proj): Linear(in_features=768, out_features=3376, bias=False)
  (norm): GraniteMoeHybridRMSNormGated()
  (out_proj): Linear(in_features=1536, out_features=768, bias=False)
)
(13): GraniteMoeHybridDecoderLayer(
  (block_sparse_moe): GraniteMoeHybridMoE(
    (activation): SiLUActivation()
    (input_linear): GraniteMoeHybridParallelExperts()
    (output_linear): GraniteMoeHybridParallelExperts()
    (router): GraniteMoeHybridTopKGating(
      (layer): Linear(in_features=768, out_features=0, bias=False)
```

```
(lm_head): Linear(in_features=768, out_features=100352, bias=False)
)

[3]: prompt="""
Please list one programming language created by Google.
You should only output its name and primary use case.
"""
chat = [{"role": "user", "content": prompt},]
chat = tokenizer.apply_chat_template(chat, tokenize=False, add_generation_prompt=True)
input_tokens = tokenizer(chat, return_tensors="pt").to(device)
output = model.generate(**input_tokens, max_new_tokens=100)
output = tokenizer.batch_decode(output)
print(output[0])

<|start_of_role|>system<|end_of_role|>You are a helpful assistant. Please ensure responses are professional, accurate, and safe.<|end_of_text|>
<|start_of_role|>user<|end_of_role|>
Please list one programming language created by Google.
You should only output its name and primary use case.
<|end_of_text|>
<|start_of_role|>assistant<|end_of_role|>One programming language created by Google is Java. Its primary use case is software development.<|end_of_text|>

[ ]:
```

```
[4]: prompt="""
Please list one search engine company founded in 1998.
You should only output its name and headquarters location.
"""

chat = [{"role": "user", "content": prompt},]
chat = tokenizer.apply_chat_template(chat, tokenize=False, add_generation_prompt=True)
input_tokens = tokenizer(chat, return_tensors="pt").to(device)
output = model.generate(**input_tokens, max_new_tokens=100)
output = tokenizer.batch_decode(output)
print(output[0])

<|start_of_role|>system<|end_of_role|>You are a helpful assistant. Please ensure responses are professional, accurate, and safe.<|end_of_text|>
<|start_of_role|>user<|end_of_role|>
Please list one search engine company founded in 1998.
You should only output its name and headquarters location.
<|end_of_text|>
<|start_of_role|>assistant<|end_of_role|>Here is one search engine company founded in 1998:

Google<|end_of_text|>
```

```
[9]: prompt="""
What is the new name of country Swaziland?
"""

chat = [{"role": "user", "content": prompt},]
chat = tokenizer.apply_chat_template(chat, tokenize=False, add_generation_prompt=True)
input_tokens = tokenizer(chat, return_tensors="pt").to(device)
output = model.generate(**input_tokens, max_new_tokens=100)
output = tokenizer.batch_decode(output)
print(output[0])

<|start_of_role|>system<|end_of_role|>You are a helpful assistant. Please ensure responses are professional, accurate, and safe.<|end_of_text|>
<|start_of_role|>user<|end_of_role|>
What is the new name of country Swaziland?
<|end_of_text|>
<|start_of_role|>assistant<|end_of_role|>The new name of the country Swaziland is Eswatini.<|end_of_text|>
```

```
[11]: prompt="""
What color do you get when you mix blue and yellow? Only provide the color name.
"""

chat = [{"role": "user", "content": prompt},]
chat = tokenizer.apply_chat_template(chat, tokenize=False, add_generation_prompt=True)
input_tokens = tokenizer(chat, return_tensors="pt").to(device)
output = model.generate(**input_tokens, max_new_tokens=100)
output = tokenizer.batch_decode(output)
print(output[0])

<|start_of_role|>system<|end_of_role|>You are a helpful assistant. Please ensure responses are professional, accurate, and safe.<|end_of_text|>
<|start_of_role|>user<|end_of_role|>
What color do you get when you mix blue and yellow? Only provide the color name.
<|end_of_text|>
<|start_of_role|>assistant<|end_of_role|>The color you get when you mix blue and yellow is green.<|end_of_text|>
```

```
[13]: prompt="""
What is 15 multiplied by 4 and then divided by 7? Only provide the number.
"""

chat = [{"role": "user", "content": prompt},]
chat = tokenizer.apply_chat_template(chat, tokenize=False, add_generation_prompt=True)
input_tokens = tokenizer(chat, return_tensors="pt").to(device)
output = model.generate(**input_tokens, max_new_tokens=100)
output = tokenizer.batch_decode(output)
print(output[0])

<|start_of_role|>system<|end_of_role|>You are a helpful assistant. Please ensure responses are professional, accurate, and safe.<|end_of_text|>
<|start_of_role|>user<|end_of_role|>
What is 15 multiplied by 4 and then divided by 7? Only provide the number.
<|end_of_text|>
<|start_of_role|>assistant<|end_of_role|>8.57<|end_of_text|>
```

```
[ ]: tools = [
    {
        "type": "function",
        "function": {
            "name": "calculate_tip",
            "description": "Calculate the tip amount for a restaurant bill.",
            "parameters": {
                "type": "object",
                "properties": {
                    "bill_amount": {
                        "type": "number",
                        "description": "Total bill amount in dollars"
                    },
                    "tip_percentage": {
                        "type": "number",
                        "description": "Tip percentage (e.g., 15, 18, 20)"
                    }
                },
                "required": ["bill_amount", "tip_percentage"]
            }
        }
    }
]

chat = [
```

```
        "required": ["bill_amount", "tip_percentage"]
    }
}

chat = [
    { "role": "user", "content": "I have a $45 restaurant bill. Calculate a 20% tip for me." },
]

chat = tokenizer.apply_chat_template(chat, tokenize=False, tools=tools, add_generation_prompt=True)
input_tokens = tokenizer(chat, return_tensors="pt").to(device)
output = model.generate(**input_tokens, max_new_tokens=100)
output = tokenizer.batch_decode(output)
print(output[0])
```

```
input_tokens = tokenizer(chat, return_tensors="pt").to(device)
output = model.generate(**input_tokens, max_new_tokens=100)
output = tokenizer.batch_decode(output)
print(output[0])

<|start_of_role|>system<|end_of_role|>You are a helpful assistant with access to the following tools. You may call one or more tools to assist with the user query.

You are provided with function signatures within <tools></tools> XML tags:
<tools>
{"type": "function", "function": {"name": "calculate_tip", "description": "Calculate the tip amount for a restaurant bill.", "parameters": {"type": "object", "properties": {"bill_amount": {"type": "number", "description": "Total bill amount in dollars"}, "tip_percentage": {"type": "number", "description": "Tip percentage (e.g., 15, 18, 20)"}}, "required": ["bill_amount", "tip_percentage"]}}}
</tools>

For each tool call, return a json object with function name and arguments within <tool_call></tool_call> XML tags:
<tool_call>
{"name": <function-name>, "arguments": <args-json-object>}
</tool_call>. If a tool does not exist in the provided list of tools, notify the user that you do not have the ability to fulfill the request.<|end_of_text|>
<|start_of_role|>user<|end_of_role|>I have a $45 restaurant bill. Calculate a 20% tip for me.<|end_of_text|>
<|start_of_role|>assistant<|end_of_role|><tool_call>
{"name": "calculate_tip", "arguments": {"bill_amount": 45, "tip_percentage": 20}}
</tool_call><|end_of_text|>
```

The model is able to correctly identify and execute the tool call.

```
[6]: GraniteMoeHybridForCausalLM(
      (model): GraniteMoeHybridModel(
        (embed_tokens): Embedding(100352, 768, padding_idx=100256)

[15]: prompt = """
Translate the following sentence into
German, Spanish, French, Japanese, Portuguese, Arabic, Czech, Italian, Korean, Dutch, Hindi, Indonesian and Chinese:
"Spend less than what you earn"
"""

chat = [{"role": "user", "content": prompt}]
chat = tokenizer.apply_chat_template(chat, tokenize=False, add_generation_prompt=True)
input_tokens = tokenizer(chat, return_tensors="pt").to(device)
output = model.generate(**input_tokens, max_new_tokens=500)
output = tokenizer.batch_decode(output)
print(output[0])

<|start_of_role|>system<|end_of_role|>You are a helpful assistant. Please ensure responses are professional, accurate, and safe.<|end_of_text|>
<|start_of_role|>user<|end_of_role|>
Translate the following sentence into
German, Spanish, French, Japanese, Portuguese, Arabic, Czech, Italian, Korean, Dutch, Hindi, Indonesian and Chinese:
"Spend less than what you earn"
<|end_of_text|>
<|start_of_role|>assistant<|end_of_role|>Here is the translation of the sentence "Spend less than what you earn" into German, Spanish, French, Japanese, Portuguese, Arabic, Czech, Italian, Korean, Dutch, Hindi, Indonesian, and Chinese:

Deutsch: "Spahren weniger als was du erhalst"
```

```
"Spend less than what you earn"
<|end_of_text|>
<|start_of_role|>assistant<|end_of_role|>Here is the translation of the sentence "Spend less than what you earn" into German, Spanish, French, Japanese, Portuguese, Arabic, Czech, Italian, Korean, Dutch, Hindi, Indonesian, and Chinese:

Deutsch: "Spahren weniger als was du erhalst"
Spanish: "Pase menos que lo que earns"
French: "Pensez moins que ce que tu gagne"
Japanese: "お投げ金少ない"
Portuguese: "Pensei menos que o que ganho"
Arabic: "ياخيار ما بتحمل"
Czech: "Přečti méně než co dostáváte"
Italian: "Pensa meno di quello che ricevi"
Korean: "눈에 띄게 소모하지 않습니다"
Dutch: "Sparen minder dan wat u kinks"
Hindi: "अपनी खर्चिता को कम करें"
Indonesian: "Pensi lebih dari yang diambil"
Chinese (Simplified): "少量的钱可以投<|end_of_text|>

[ ]:
```

Language recognition works but the translation fails

```
[6]: GraniteMoeHybridForCausalLM(
      (model): GraniteMoeHybridModel(
        (embed_tokens): Embedding(100352, 768, padding_idx=100256)

[16]: prompt = """
Translate the following sentence into
German, Spanish, French, Japanese, Portuguese, Arabic, Czech, Italian, Korean, Dutch, Hindi, Indonesian and Chinese:
"how much is this?"
"""

chat = [{"role": "user", "content": prompt}]
chat = tokenizer.apply_chat_template(chat, tokenize=False, add_generation_prompt=True)
input_tokens = tokenizer(chat, return_tensors="pt").to(device)
output = model.generate(**input_tokens, max_new_tokens=500)
output = tokenizer.batch_decode(output)
print(output[0])

<|start_of_role|>system<|end_of_role|>You are a helpful assistant. Please ensure responses are professional, accurate, and safe.<|end_of_text|>
<|start_of_role|>user<|end_of_role|>
Translate the following sentence into
German, Spanish, French, Japanese, Portuguese, Arabic, Czech, Italian, Korean, Dutch, Hindi, Indonesian and Chinese:
"how much is this?"
<|end_of_text|>
<|start_of_role|>assistant<|end_of_role|>Here is the translation of the sentence "how much is this?" in the requested languages:

German: Wie viel ist das?
```

```
chat = [{"role": "user", "content": prompt}]
chat = tokenizer.apply_chat_template(chat, tokenize=False, add_generation_prompt=True)
input_tokens = tokenizer(chat, return_tensors="pt").to(device)
output = model.generate(**input_tokens, max_new_tokens=500)
output = tokenizer.batch_decode(output)
print(output[0])
```

Translate the following sentence into
German, Spanish, French, Japanese, Portuguese, Arabic, Czech, Italian, Korean, Dutch, Hindi, Indonesian and Chinese:
"how much is this?"
<|end_of_text|>
<|start_of_role|>assistant<|end_of_role|>Here is the translation of the sentence "how much is this?" in the requested languages:

German: Wie viel ist das?

Spanish: ¿Cuánto es esto?

French: Combien ça ça est?

Japanese: これはいくらですか?

Portuguese: Como é este?

Arabic: كم هذا<|end_of_text|>

These translations are mostly correct but not all.